

MIT 6.86x: 오차를 통해 배우다

Unit 4: Backpropagation & Optimization Details

Contents

1	Unit 4. 역전파와 학습의 원리 (Backpropagation)	2
1.1	1. 역전파 알고리즘 (Backpropagation)	2
1.2	2. 손실 함수 (Loss Function)	3
1.3	3. 초기화와 학습률 (Initialization & Learning Rate)	3
1.4	4. 학습 안정화 기법: 배치 정규화 & 드롭아웃	4
2	실전 시나리오: 넥슨 게임 이탈(Churn) 방지 모델	5
3	자주 묻는 질문 (FAQ)	5

Course Structure & Current Focus

- Unit 3: Neural Networks Basics (구조 설계)
- **Unit 4: Backpropagation (현재 단원: 학습 메커니즘)**
 - 8.1 The Engine: Backpropagation Algorithm
 - 8.2 The Scorecard: Loss Functions (MSE vs Cross-Entropy)
 - 8.3 Setting the Stage: Initialization & Learning Rate
 - 8.4 Stabilization: Batch Norm & Dropout
- Unit 5: Recurrent Neural Networks (순차 데이터)

1 Unit 4. 역전파와 학습의 원리 (Backpropagation)

Unit 3에서 우리는 수천 개의 뉴런이 연결된 뇌 (FNN)를 만들었습니다. 하지만 갓 태어난 뇌는 아는 것이 없습니다(가중치 W 가 랜덤임). 이제 이 뇌에게 정답을 보여주고, 틀릴 때마다 "누가 잘못했는지"를 따져서 조금씩 똑똑해지게 만들어야 합니다. 이 과정이 바로 학습 (Training)입니다.

□ 개요 (Overview)

이 단원에서는 신경망 학습의 핵심 엔진인 역전파(Backpropagation) 알고리즘을 배웁니다. 미분의 연쇄 법칙(Chain Rule)을 이용해 오차의 원인을 찾아내고, 손실 함수(Loss Function)를 통해 성적을 매기며, 초기화(Initialization)와 정규화(Normalization) 기법으로 학습을 안정화시키는 전체 파이프라인을 다룹니다.

□ 핵심 용어 사전

용어 (Term)	직관적 의미 (Meaning)
Forward Pass	입력 → 출력. 예측값을 뽑아내는 과정.
Backward Pass	출력 → 입력. 틀린 이유(미분값)를 배달하는 과정.
Chain Rule	꼬리에 꼬리를 무는 미분. (나비효과 추적)
Epoch	전체 문제집을 한 번 다 풀어보는 것.
Dropout	일부 뉴런을 랜덤하게 꺼서, 특정 뉴런 편애를 막는 기술.

1.1 1. 역전파 알고리즘 (Backpropagation)

□ 개념 1: 책임 소재 따지기 (The Blame Game)

한 줄 요약: 결과가 틀렸을 때, 출력층에서부터 거꾸로 거슬러 올라가며 각 뉴런이 오차에 기여한 만큼(기울기) 가중치를 수정합니다.

1) 작동 원리: 연쇄 법칙 (Chain Rule)

우리는 최종 오차 L 을 줄이기 위해 가중치 w 를 조절하고 싶습니다. 즉, $\frac{\partial L}{\partial w}$ 가 필요합니다. 하지만 L 과 w 는 멀리 떨어져 있습니다.

$$\frac{\partial L}{\partial w} = \underbrace{\frac{\partial L}{\partial y}}_{\text{오차 변화}} \cdot \underbrace{\frac{\partial y}{\partial h}}_{\text{출력 변화}} \cdot \underbrace{\frac{\partial h}{\partial w}}_{\text{은닉층 변화}}$$

- 마치 통역을 거치듯, 미분값을 층층이 곱해서 전달합니다.
- **** (위 이미지 참고): δ 값(오차 신호)이 오른쪽(a^4)에서 왼쪽(a^0)으로 전파되는 것을 볼 수 있습니다.

2) 숫자 예시

간단한 식 $L = (y - wx)^2$ 에서, $y = 10, x = 2, w = 3$ 이라고 합시다.

1. 순전파: 예측 $\hat{y} = 3 \times 2 = 6$.
2. 오차: $L = (10 - 6)^2 = 16$.
3. 역전파: w 를 조절하면 L 이 얼마나 변할까?

$$\frac{\partial L}{\partial w} = 2(y - \hat{y}) \cdot (-x) = 2(4) \cdot (-2) = -16$$

4. 의미: 기울기가 음수(-16)이므로, w 를 키워야 오차가 줄어듭니다.

1.2 2. 손실 함수 (Loss Function)

□ 개념 2: 채점 기준표

한 줄 요약: 문제의 유형(회귀 vs 분류)에 따라 "얼마나 틀렸는지"를 계산하는 방식이 다릅니다.

구분	MSE (Mean Squared Error)	Cross-Entropy
용도	회귀 (집값, 온도 예측)	분류 (스팸, 아이템 등급)
수식	$\frac{1}{n} \sum (y - \hat{y})^2$	$-\sum y \log(\hat{y})$
특징	거리가 멀수록 제곱으로 혼냄.	확률이 틀릴수록 급격히 혼냄.

왜 분류 문제에 MSE를 안 쓰나요?

Sigmoid + MSE 조합을 쓰면, 오차가 엄청 큰데도 미분값(Gradient)이 0에 가까워져서 학습이 안 되는 현상이 발생합니다. 반면 **Cross-Entropy**는 로그 함수 특성상 틀리면 미분값이 폭발적으로 커져서 "정신 차려!"라고 강하게 피드백을 줍니다.

1.3 3. 초기화와 학습률 (Initialization & Learning Rate)

□ 개념 3: 시작이 반이다

한 줄 요약: 출발점(초기화)을 잘 잡고, 보폭(학습률)을 적절히 조절해야 산 아래로 잘 내려갈 수 있습니다.

1) 가중치 초기화 (Initialization)

- **0으로 초기화하면? (The Trap of Symmetry):** 모든 뉴런이 똑같은 값을 내뱉고, 똑같이 업데이트됩니다. 100개를 써도 1개 쓰는 것과 같습니다.
- **해결책:**
 - **He Initialization:** ReLU 함수를 쓸 때 표준. (분산을 $2/n$ 로 맞춤)
 - **Xavier Initialization:** Sigmoid/Tanh를 쓸 때 표준.

2) 학습률 스케줄링 (Learning Rate Decay)

처음엔 성큼성큼(큰 η) 가다가, 정답 근처에서는 조심조심(작은 η) 가야 합니다.

$$\eta_t = \eta_0 / (1 + kt) \quad (\text{시간이 갈수록 줄어듦})$$

1.4 4. 학습 안정화 기법: 배치 정규화 & 드롭아웃

□ 개념 4: 흔들리지 않는 편안함

한 줄 요약: 딥러닝은 깊어질수록 학습이 불안정해집니다. 이를 막기 위해 데이터를 강제로 정렬하거나(Batch Norm), 일부러 핸디캡을 줍니다(Dropout).

A. 배치 정규화 (Batch Normalization)

- **문제:** 앞단 레이어의 가중치가 바뀌면, 뒷단 레이어 입장에서는 들어오는 데이터 분포가 계속 바뀝니다 (Internal Covariate Shift).
- **해결:** 각 층마다 들어오는 데이터를 **평균 0, 분산 1**로 강제 조정해 버립니다.
- **효과:** 학습 속도가 비약적으로 빨라지고, 초기화에 덜 민감해집니다.

B. 드롭아웃 (Dropout)

- **개념:** 학습할 때마다 은닉층 뉴런의 50%를 랜덤하게 꺼버립니다.
- **비유:** 축구팀에서 에이스(특정 뉴런) 한 명만 믿는 것을 방지하기 위해, 연습 때 에이스를 벤치에 앉혀두고 나머지 선수들끼리 훈련시킵니다. 팀 전체 전력이 올라 갑니다(과적합 방지).

2 실전 시나리오: 넥슨 게임 이탈(Churn) 방지 모델

□ Scenario: VIP 유저가 떠나는 이유 찾기

당신은 넥슨의 PM입니다. "이번 달에 접속할 것"이라고 예측(0.9)했던 VIP 유저가 실제로는 접속하지 않았습니다(Target 0.0). 오차가 큼니다.

1. **Forward:** 유저 데이터(길드 활동, 결제액 등) → FNN → 예측확률 90%.
2. **Loss:** Cross-Entropy로 계산하니 손실이 매우 큼니다. ("확실했는데 틀렸군!")
3. **Backward:**
 - 역전파가 시작됩니다. 출력층에서 은닉층으로 거슬러 올라갑니다.
 - 범인 색출: "길드 활동 점수"에 연결된 가중치가 범인이었습니다. "길드 활동이 많으면 무조건 남는다"고 과대평가하고 있었네요.
4. **Update:** 해당 가중치를 대폭 삭감합니다.
5. **Result:** 모델은 이제 "길드 활동이 많아도 최근 접속이 뜸하면 이탈할 수 있다"는 미묘한 패턴을 배우게 됩니다.

3 자주 묻는 질문 (FAQ)

- Q1. 역전파를 손으로 계산할 줄 알아야 하나요?** A. 개념만 알면 됩니다. PyTorch나 TensorFlow 같은 프레임워크가 'loss.backward()' 한 줄이면 자동으로 미분을 다 해줍니다. (이걸 Autograd라고 합니다.)
- Q2. Dropout은 테스트(실전) 때도 쓰나요?** A. 절대 아닙니다! 학습(Training) 때는 뉴런을 끄지만, 테스트(Inference) 때는 모든 뉴런을 켜서 전력을 다해야 합니다. 대신 출력값에 비율(0.5)을 곱해서 보정해 줍니다.

Next Step: 지금까지 배운 신경망은 정지된 이미지나 테이블 데이터에는 잘 작동하지만, "문장"이나 "주가"처럼 순서가 있는(Sequential) 데이터에는 약합니다. 다음 Unit 5에서는 기억력(Memory)을 가진 신경망, RNN(Recurrent Neural Networks)과 LSTM을 배웁니다.

Unit 4 핵심 요약

- **역전파 (Backprop):** Chain Rule을 이용해 오차의 책임 (Gradient)을 각 가중치에 배분한다.
- **손실 함수:** 회귀는 MSE, 분류는 Cross-Entropy가 국룰이다.
- **He Initialization:** ReLU를 쓸 때는 초기화를 잘해야 학습이 시작된다.
- **Batch Norm & Dropout:** 학습을 빠르고 안정적으로 만들고 과적합을 막는 필수テクニック.